

PROJET DE FIN D'ANNÉE (PFA)

Plateforme DevOps pour l'Automatisation du Cycle de Vie d'une Application Web à Architecture Microservices

Application : Plateforme de Réservation (salles, événements, rendez-vous)

Université	Université Hassan 1er
Établissement	Faculté des Sciences et Techniques — Settat
Réalisé par	Mouine Aya
Encadrant	M. TOUFIK Fouad
Année universitaire	2025-2026

Table des matières

- 1. Contexte et cahier des charges
- 2. Architecture microservices
- 3. Réalisation (backend, frontend, base de données)
- 4. DevOps (Docker, Kubernetes, CI/CD, Monitoring)
- 5. Tests et validation
- 6. Résultats chiffrés
- 7. Difficultés rencontrées
- 8. Limites et perspectives
- 9. Conclusion
- 10. Annexes

1. Contexte et cahier des charges

1.1 Problématique

La Faculté des Sciences et Techniques de Settat dispose de salles, d'amphithéâtres, de clubs et d'événements dont la gestion manuelle des réservations (papier / tableur) est source de conflits de créneaux, de pertes de temps et d'une absence totale de traçabilité.

Le cahier des charges définit une **plateforme web décentralisée** permettant aux différents acteurs de l'établissement d'effectuer leurs réservations, de les faire valider selon leur rôle, d'être notifiés de l'évolution, et de régler les frais associés — le tout de façon sécurisée et fiable.

1.2 Besoins fonctionnels

- Authentification et gestion des comptes par rôle ;
- Consultation des salles et de l'emploi du temps ;
- Demande, validation (3 niveaux), suivi et paiement des réservations ;

- Gestion des événements (clubs) avec pièce jointe PDF et calendrier ;
- Notifications aux parties prenantes ;
- Gestion administrative des salles, utilisateurs et ressources.

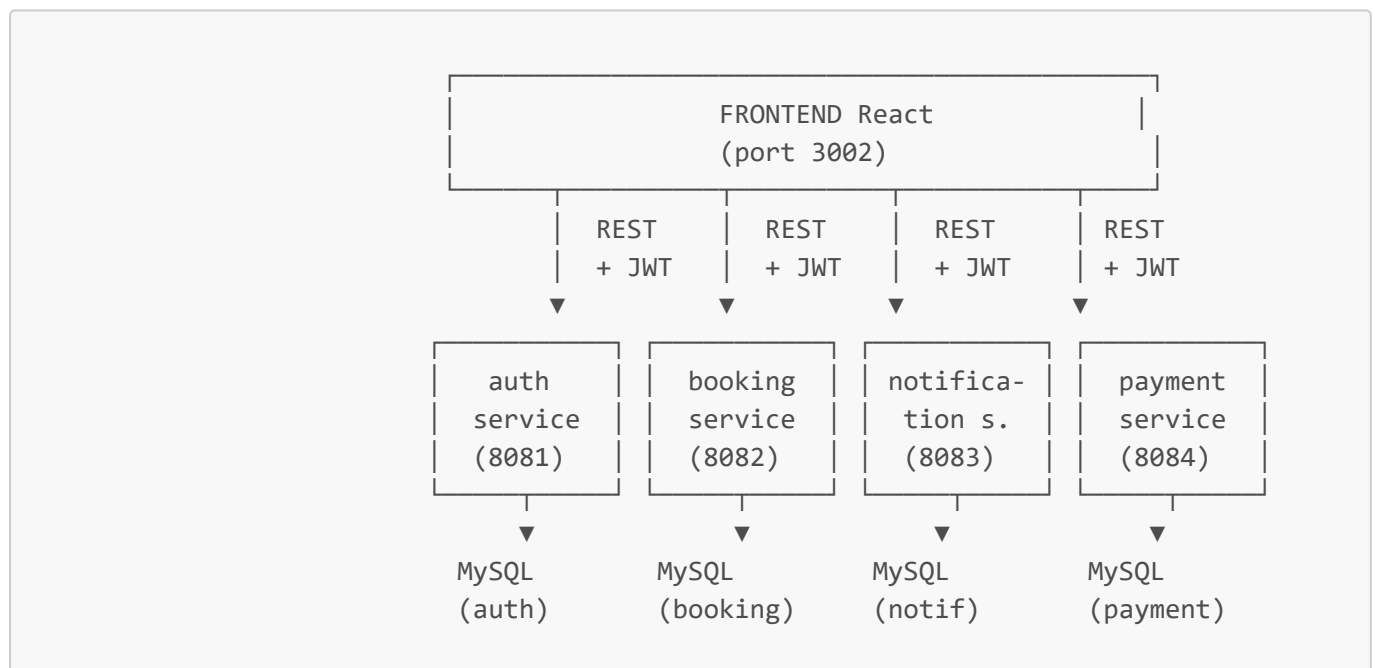
1.3 Besoins non fonctionnels

- **Scalabilité** : architecture microservices, déployable en cluster ;
- **Disponibilité** : orchestration Kubernetes, redémarrage automatique, probes ;
- **Cohérence** : anti double-réservation (contrainte DB + test de concurrence) ;
- **Sécurité** : JWT, BCrypt, contrôle des rôles, secret JWT externalisé ;
- **Performance** : temps de réponse moyens satisfaisants, supervision active ;
- **Maintenabilité** : code modulaire, documentation et diagrammes UML ;
- **Portabilité** : conteneurisation Docker, portage Kubernetes et kind.

2. Architecture microservices

2.1 Vue d'ensemble

Quatre microservices Spring Boot partagent avec le frontend React le cycle de vie complet d'une réservation. Chaque service possède **sa propre base MySQL** (principe d'indépendance des données), et les appels inter-services sont authentifiés par JWT.

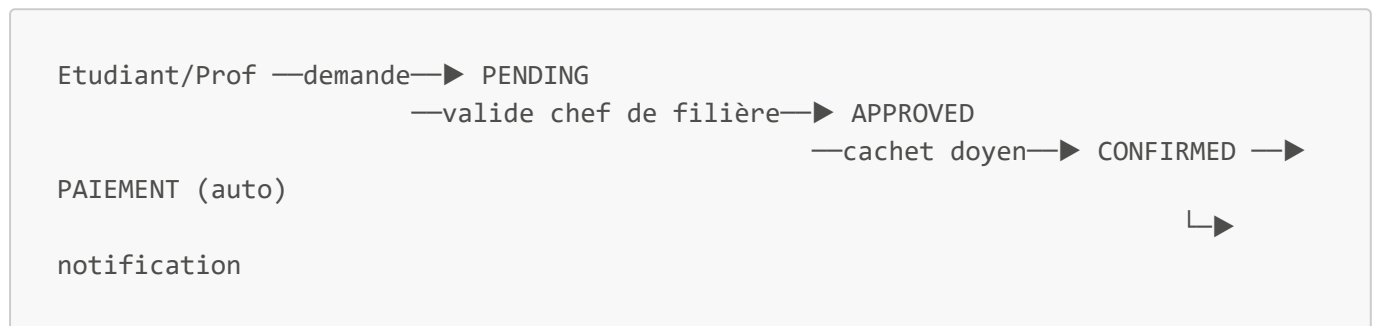


2.2 Services et ports

Service	Rôle	Port interne	Port exposé
auth-service	Inscription, login, JWT, rôles	8081	18081
booking-service	Salles, créneaux, workflow de réservation, PDF	8082	8082

Service	Rôle	Port interne	Port exposé
notification-service	Notifications ciblées (rôle / filière / utilisateur)	8083	18083
payment-service	Paieement simulé, déclenché au cachet du doyen	8084	8084
frontend	Application React (Router, Axios, Context auth)	3000	3002

2.3 Workflow de réservation (3 niveaux)



Un **booking** ne peut exister qu'avec un créneau libre : contrainte **UNIQUE(salle, date, créneau)** + vérification applicative, ce qui empêche les doubles réservations.

2.4 Rôles

Rôle	Droits
ETUDIANT	Consulte les salles / emploi du temps (lecture seule)
PROF	Demande, modifie, annule ; joint un PDF
CHEF_FILIERE	Valide/refuse les demandes de sa filière
DOYEN	Cachet final + déclenche le paiement
CLUB	Crée des événements avec pièce jointe et calendrier
ADMIN	Gère les salles et les utilisateurs ; consulte le tableau de bord

2.5 Choix techniques justifiés

- **Spring Boot** : socle Java standard des microservices (Actuator, sécurité, validation).
- **Spring Security + JWT + BCrypt** : sécurité sans état (adaptée à des API et à un cluster).
- **MySQL** : SGBD relationnel répandu, contraintes **UNIQUE** pour l'absence de doublons.
- **React (CRA)** : interface modulaire, routage par rôle.
- **Docker + Docker Compose** : portabilité et environnement local reproductible.
- **Kubernetes / kind** : orchestration, auto-réparation, déploiement identique CI/production.
- **Prometheus + Grafana** : supervision des métriques des 4 services.
- **GitHub Actions** : CI/CD automatisée (build, tests, image, déploiement kind, E2E).

3. Réalisation

3.1 Backend — 4 microservices Spring Boot

Chaque service suit la même ossature :

```
config (sécurité, data, seed) → controller (REST) → service (logique métier)
    → repository (JPA) → entité JPA → base MySQL dédiée
```

- **auth-service** : **AuthController** (login, inscription, refresh, mot de passe oublié), filtre JWT, **UserDataLoader** seedant 5 rôles + 6 clubs, validation du format d'email institutionnel (regex), BCrypt.
- **booking-service** : **BookingController** — salles, créneaux, workflow 3 niveaux, génération de PDF, résolution de prix, déclenchement du paiement au cachet du doyen, tests de concurrence.
- **notification-service** : **NotificationController** — ciblage **par rôle, par filière et par utilisateur** (endpoint `/my` résolu depuis le JWT), statuts lus/non lus, préférences.
- **payment-service** : **PaymentController** — paiement simulé déclenché automatiquement à la confirmation du doyen, journal des paiements, endpoints de supervision.

Sécurité inter-services : chaque appel entre services porte un **JWT de service** vérifié par le récepteur ; les mots de passe sont hachés avec **BCrypt**.

3.2 Frontend — React

- SPA React (Create React App), routage par rôle, Context pour l'authentification, Axios avec intercepteur JWT.
- Pages : accueil public (FST Settat), inscription, connexion, salles, emploi du temps, réservation, suivi, notifications, gestion des événements, tableau de bord admin.
- Affichage conditionnel des actions selon le rôle (bouton "cachet du doyen", "valider", "paiement" selon le statut).

3.3 Base de données

Un schéma MySQL par service (indépendance des données) :

Service	Base	Contrainte clé
auth	auth_db	UNIQUE(username), UNIQUE(email)
booking	booking_db	UNIQUE(salle, date, créneau) anti doublon
notification	notification_db	FK vers utilisateur, statut ENUM
payment	payment_db	UNIQUE(booking_id) — un seul paiement par réservation

Anti double-réservation : contrainte SQL **UNIQUE** + test de concurrence (deux réservations simultanées sur le même créneau → une seule est retenue).

3.4 Conteneurisation

- **Dockerfile** multi-stage par service (cache Maven, image JRE légère) et pour le frontend (build Node → serveur statique Nginx).
- **docker-compose.yml** : MySQL (×4), les 4 services, le frontend, **phpMyAdmin (8085)**, **Prometheus (9090)** et **Grafana (3000)** avec volumes pérennes.
- Health-checks applicatifs par service (Actuator).

4. DevOps — Docker, Kubernetes, CI/CD, Monitoring

4.1 Docker Compose (environnement local)

```
docker compose up -d --build
```

Accès : frontend <http://localhost:3002>, phpMyAdmin <http://localhost:8085>, Prometheus <http://localhost:9090>, Grafana <http://localhost:3000>.

4.2 Kubernetes (kind) — orchestration

- Manifests **k8s/** : MySQL, 4 services, frontend, Prometheus, Grafana, Secret JWT ;
- **Deployment** avec **replicas**, **probes** (liveness/readiness) et redémarrage automatique ;
- **Service ClusterIP** par composant ; exposition du frontend et de Grafana ;
- Le pipeline CI déploie sur un cluster **kind** éphémère et valide le rollout.

4.3 CI/CD — GitHub Actions

Pipeline **ci-cd.yml**, déclenché à chaque push/sur **main** :

1. **mvn -B verify** sur les 4 services (unitaires + intégration + tests de concurrence) ;
2. Build du frontend React (lint + tests) ;
3. Build des images Docker des 5 composants (cache BuildKit) ;
4. Déploiement sur cluster **kind** : rollouts + health-checks de tous les composants + Prometheus + Grafana ;
5. Tests **E2E Cypress** (workflow complet) ;
6. Publication des images (GHCR) et supervision.

4.4 Supervision — Prometheus + Grafana

- **Actuator + Micrometer** sur chaque service : exposition </actuator/prometheus> ;
- Prometheus scrape les 4 services (**prometheus.yml**) ;
- Grafana : datasource provisionnée + dashboard « Réservation — Supervision » (memory, CPU, requêtes/s, heap JVM, threads, statut des services).

5. Tests et validation

5.1 Tests unitaires et d'intégration (Maven)

Service	Nombre de tests	Statut
auth-service	23	✓ réussis
booking-service	17	✓ réussis (dont concurrence)
notification-service	9	✓ réussis
payment-service	5	✓ réussis
Total	54	✓

`mvn -B verify` → **BUILD SUCCESS** sur les 4 services.

5.2 Tests de bout en bout (Cypress)

Scénarios E2E couvrant le workflow complet 3 niveaux (réservation, validation chef de filière, cachet du doyen, paiement, notification) → **6/6 verts** sur la stack Docker Compose (job `e2e`).

5.3 Déploiement Kubernetes (kind) validé en CI

Chaque push `main` déploie un cluster kind : tous les Deployment atteignent **Available**, les 4 services + frontend répondent `/health`, et les health-checks Prometheus/Grafana passent (job `deploy-kind`).

5.4 Sécurité

- Mots de passe hachés (BCrypt), JWT signé (secret externalisé via Secret k8s / env) ;
- Filtre d'authentification sur chaque service, whitelist contrôlée ;

6. Résultats chiffrés

- **4 microservices** + frontend React, orchestrés en Docker/K8s ;
- **54 tests** JUnit verts + **6 tests E2E** Cypress verts ;
- **CI/CD** : 12 jobs + E2E, run complet vert sur chaque push ;
- **Supervision** : Prometheus + Grafana actifs (local et k8s) ;
- **Manifests** : `k8s/` complets (MySQL, 4 services, frontend, Prometheus, Grafana).

7. Limites et perspectives

- Paiement **simulé** (pas de vraie passerelle) ;
- Notifications **en base + log** (pas d'e-mail réel SMTP) ;
- Pas de facture PDF signée chiffrée au niveau du cachet ;
- Les images GHCR sont publiques pour la démo (auth à restreindre en production) ;
- MySQL de démo sans PVC persistant (éphémère en CI).

Perspectives : e-mail réel, facture PDF signée, tests de charge sur l'anti double-réservation, alerting (Alertmanager) et dashboards Grafana avancés, mise en production sur un cluster réel.

Conclusion

La plateforme répond au cahier des charges : une application décentralisée, sécurisée et supervisée, permettant le cycle complet de réservation (demande → validation 3 niveaux → paiement → notification), conçue selon les bonnes pratiques (microservices, Conteneurisation, CI/CD, Orchestration, Supervision). Le projet est fonctionnel, testé (unitaires + E2E) et déployable de façon reproductible (Docker Compose ou Kubernetes), validant l'ensemble des besoins fonctionnels et non fonctionnels du cahier des charges.

Annexes

A.1 Comptes de démonstration

Rôle	Nom d'utilisateur	Mot de passe	Personne
ADMIN	admin	admin123	Salma El Idrissi
DOYEN	doyen	doyen123	Karim Benali
CHEF_FILIERE	chef	chef123	Nadia Alaoui (GI)
PROF	prof	prof123	Youssef Tazi (GI)
ETUDIANT	etudiant	etudiant123	Imane Rachidi (GI)
CLUB	club.<slug> (clic, ctde, bac, btec, fire, code)	club123	Club

A.2 URLs d'accès (docker compose)

Composant	URL
Frontend	http://localhost:3002
Auth API	http://localhost:18081
phpMyAdmin	http://localhost:8085
Prometheus	http://localhost:9090
Grafana	http://localhost:3000

A.3 Lancement

```
docker compose up -d --build
mvn -f services -B verify          # 4 services
cd frontend && npm run build
```

Fin du rapport final.